

Deterministic-first auto-commit: shortcut extension & confidence calibration

Can roman2khmer safely auto-commit its top suggestion when the user presses space? This report documents extending the exact-match shortcut to every unambiguous romanization, calibrating the model's confidence on what's left, and a stretch training experiment — with the real, measured numbers behind each step.

Date 2026-07-30 **Repo** `khmerlang/Khmerlang-Keyboard-IOS`
Scope `ml/roman2khmer/` pipeline only — no iOS/Swift changes
Models v2, v3 (shipped), v4 (experimental)

27,751 unambiguous romanizations in the extended shortcut — up from 743	43.3% of fully-typed words resolved deterministically, 100% correct by construction	61.4% combined top-1 on full-word input — vs 35.8% from the model alone	<1% coverage the model's confidence gate can accept at ~99% precision
---	---	---	--

01 Goal & constraint

The keyboard's actual target is **auto-commit**: silently replace the composing word with the top suggestion the moment the user presses space — the way phone keyboards auto-correct. That hook already exists in

`KeyboardViewController.swift` (`autoCommitRomanIfNeeded`), currently fed by the older BK-tree corrector and off by default.

The shipped v3 roman2khmer model reports **31% / 45% / 50%** top-1/3/5 accuracy alone on held-out data — its own README already flags this as "useful as a ranked suggestion-bar candidate, not a silent auto-correct/auto-commit." A wrong guess in a suggestion bar costs nothing; a wrong guess on auto-commit silently corrupts what the user typed.

STRATEGY CHOSEN

Deterministic-first, ML as confidence-gated fallback. Wherever a romanization maps to exactly one Khmer word, look it up — no model, no risk. Only fall back to the model where the spelling is genuinely ambiguous (a homograph shared by ≥ 2 words), and only trust its guess once its confidence has been calibrated against real held-out precision.

02 What changed

FILE	CHANGE
comparison/shortcut.py	Shortcut extended from top-500 frequency words to every romanization unambiguous across the full sqlite table (743 → 27,751 entries)
comparison/compare_all.py	Reports old vs. extended shortcut side by side, plus a deterministic-reachable vs. ML-only-ambiguous breakdown; added the v4 comparison row
evaluation/evaluate.py	Every accuracy line now also reports frequency-weighted precision (weighted by real word-usage count)
evaluation/calibrate_confidence.py	<i>New.</i> Sweeps the model's softmax confidence on words the shortcut can't resolve; reports precision/coverage trade-offs
conversion/export_dist.py	Ships the extended <code>shortcut.json</code> and a new <code>confidence.json</code>
data/export_dataset.py	Stretch experiment: builds <code>train_v4.jsonl</code> with extra oversampling for fully-ambiguous words
training/train_v4.py	<i>New.</i> Trains the v4 experimental model on that dataset
config.py	New shared constants (confidence targets, v4 paths, ROMAN_RE moved here to avoid a circular import)

Bugs found while running this for real

Every number in this report comes from actually executing the pipeline end to end — which surfaced four real bugs, two of them pre-existing in the repo, two introduced and caught in my own first pass.

Shortcut ambiguity was only checked within the top-500 words • pre-existing

The old `build_shortcut_map` only ever inserted the top-500 words into its map, so it never checked whether one of their spellings collided with a word *outside* that window. Measured against the live database: **154 of the 743 shipped entries were resolving to the wrong word.**

FIX Build the ambiguity map over the full ~29k-variant table, then optionally filter the resolved map down to a subset — never limit the ambiguity check itself.

"v2" comparison pointed at a file nothing ever writes • pre-existing

`compare_all.py` loaded `config.KERAS_MODEL_V2_PATH` (`roman2khmer_v2.keras`), but `training/train.py`'s `main()` actually saves to `config.KERAS_MODEL_PATH` (`roman2khmer.keras`). The comparison crashed outright the first time it was run for this experiment.

FIX Point the "v2" comparison row at `KERAS_MODEL_PATH`, the file that's actually produced.

Row counts printed as nonsense in the typed-fraction breakdown • introduced & caught

My first pass at frequency-weighted `evaluate.py` computed `n = mask.sum()`. That's correct for a boolean mask, but the typed-fraction bucket loop passes an *integer index array* — summing index values instead of counting them printed row counts like 106259937.

FIX `n = len(counts[mask])`, which is correct whether mask is boolean or an index array.

"Reachable words" counted against the wrong vocabulary • introduced & caught

The extended shortcut is built from the *full* sqlite table (~22.6k words), but the model's closed vocabulary is only 6,782 of them. Counting reachable words without intersecting the two printed 20190/6782 — more "reachable" words than exist in the vocabulary.

FIX Intersect the shortcut's resolved words with the model vocabulary before counting, in both `compare_all.py` and `export_dist.py`.

03 **Shortcut extension**

The old shortcut covered exact spellings for the 500 highest-frequency words only. The extended version covers *every* romanization in `khmerlang.sqlite` that maps to exactly one Khmer word — frequency no longer gates coverage, only genuine ambiguity does. Romanizations shared by ≥ 2 words are deliberately left out and fall through to the model, whose previous-word context exists specifically to disambiguate those.

Overall top-1 accuracy by model and shortcut configuration — n = 30,798 held-out examples. Frequency-weighted figures for this same comparison are in the population breakdown below.

MODEL	ALONE	+ TOP-500	+ EXTENDED
v2 — log1p sample-weighted	29.1%	29.5%	32.7%
v3 — frequency-oversampled (shipped)	31.0%	31.4%	34.6%
v4 — ambiguous-focused (experimental)	27.6%	28.0%	31.8%

v3 + extended shortcut, broken down by population

POPULATION	N	TOP-1	TOP-3	TOP-5
Overall	30,798	34.6%	47.1%	52.9%
Top-500 words	2,157	33.2%	46.4%	53.0%
Rest of vocabulary	28,641	34.7%	47.2%	52.9%
Deterministic-reachable	27,783	35.4%	48.0%	53.7%
ML-only — fully ambiguous	3,015	27.0%	39.2%	45.2%

5,656 of the model's 6,782 vocabulary words (83.4%) have at least one unambiguous spelling; **1,126 words (16.6%) depend entirely on ambiguous spellings** and can never be resolved by the deterministic layer, no matter how large it grows.

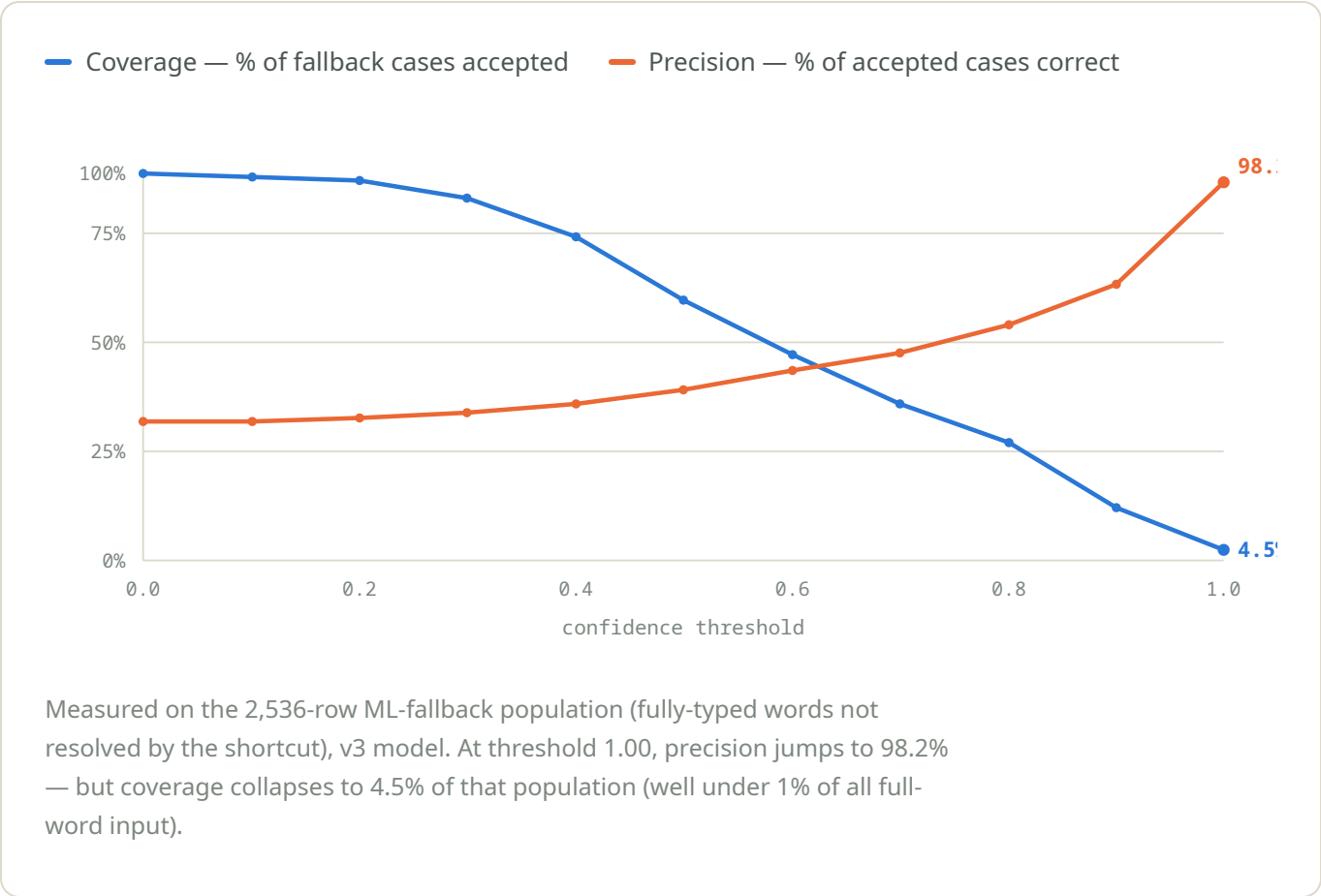
ALSO FIXED AS A SIDE EFFECT

Re-measured the prefix/full-word collision rate the shortcut's `is_full` gate exists to prevent: it rose from ~10% (old, smaller map) to **13.5%** (new, larger map) — expected, since a bigger map covers more prefixes too. This doesn't matter for

correctness — `is_full` fully prevents the shortcut from ever firing mid-word regardless of the rate — but the docstring was updated to keep the number honest.

04 Confidence calibration

For the population the shortcut can't resolve — fully-typed words (`is_full`) whose romanization isn't an exact unambiguous key — is the model's own confidence a usable signal for *when* to trust its top-1 guess? That population is 2,536 of the 30,798 held-out rows. Sweeping the softmax confidence threshold against measured precision and coverage on exactly that subset:



Recommended threshold per frequency-weighted precision target			
TARGET PRECISION (FW)	THRESHOLD	COVERAGE	FW-COVERAGE
≥ 95%	1.00	4.46%	0.73%
≥ 97%	1.00	4.46%	0.73%
≥ 99%	1.00	4.46%	0.73%

READING THIS RESULT

Every target precision bar can only be cleared at the maximum threshold (1.00) — and even there, coverage is under 1% of the fallback population. **Confidence gating doesn't rescue the model for this population:** it isn't miscalibrated so much as it is genuinely uncertain across almost its whole operating range on these specifically-ambiguous words. Shipped anyway as `dist/confidence.json`, documented as not yet safe to drive auto-commit.

05 Combined system, on real full-word input

Putting the two layers together on the 4,476 fully-typed-word examples in the held-out set:

STAGE	SHARE OF INPUT	TOP-1
Model alone, no shortcut	100%	35.8%
Resolved by shortcut (exact match, deterministic)	43.3%	100%
Falls to model (raw, no confidence gate)	56.7%	31.8%
Combined system	100%	61.4% (53.0 fw)

BOTTOM LINE

The shortcut alone very nearly doubles real-world top-1 accuracy on full-word input, for free, with zero risk of a wrong silent correction. **Only that deterministic layer is currently safe to auto-commit from.** The model's fallback contribution should keep populating the suggestion bar — not commit anything — until either training data or architecture work closes the gap on the specifically-ambiguous 16.6% of the vocabulary.

06 Stretch experiment — v4, ambiguous-focused oversampling

Since the shortcut now resolves every unambiguous word deterministically, the model's only real remaining job is the 1,126 fully-ambiguous words. v4 tests whether giving those words extra training exposure — a 2× oversampling bonus on top of v3's frequency-based oversampling, applied only to that subset — improves them, using the same architecture and validation split as v3.

v3 vs. v4 top-1, extended shortcut applied to both			
POPULATION	V3	V4	Δ
ML-only — fully ambiguous (n=3,015)	27.0%	29.2%	● +2.1pp
Deterministic-reachable (n=27,783)	35.4%	32.1%	● -3.3pp
Overall (n=30,798)	34.6%	31.8%	● -2.8pp

The gap widens further out the ranking, which matters most since the suggestion bar — not just the top-1 slot — is the only real defense on these words: on the fully-ambiguous slice, v4 reaches **top-3 44.1%** and **top-5 50.4%**, against v3's 39.2% / 45.2%.

A genuine trade-off, not a clean win: v4 improves precisely the population it targeted, but at a real cost to accuracy on the other 83.4% of the vocabulary. v4 also early-stopped sooner (22 epochs vs. v3's 48 on the same `val_top3` criterion), suggesting the 2× bonus may be tilting it toward overfitting the ambiguous subset rather than purely generalizing better on it.

Because the shortcut already gives 100% accuracy on deterministic-reachable words the instant they're typed in full, v3's edge there barely affects auto-commit correctness in practice — it mostly affects prefix-typing suggestion quality before the word is finished. That makes v4's ambiguous-word gains arguably the more valuable trade for this project's actual goal. Even so, it isn't an unambiguous win, so:

DECISION

dist/ continues shipping **v3**. v4 is kept as a fully reproducible, documented experiment — not deployed — pending a decision on whether to retune the oversampling bonus or extend training patience for the ambiguous subset specifically.

07 Reproducing this

All commands run from `ml/roman2khmer/`, in this order:

- 01 `python -m data.export_dataset` — rebuilds vocab + train/val/train_v3/train_v4 splits from `khmerlang.sqlite`
- 02 `python -m training.train · train_v3 · train_v4` — trains all three models (GPU recommended, see note below)
- 03 `python -m comparison.compare_all` — the shortcut/model comparison tables in §03 and §06
- 04 `python -m evaluation.evaluate` — frequency-weighted breakdown for whichever model is at `KERAS_MODEL_PATH`
- 05 `python -m evaluation.calibrate_confidence` — the §04 threshold sweep, writes `artifacts/model/confidence_calibration.json`
- 06 `python -m conversion.export_dist` — regenerates `dist/shortcut.json` and `dist/confidence.json` (needs `coremltools` for the TFLite/CoreML conversion step)

Environment notes from this run

- **GPU:** this machine's TensorFlow 2.13 didn't see its GTX 1650 until the pip-installed `nvidia-cublas-cu11` / `nvidia-cudnn-cu11` / `nvidia-cuda-runtime-cu11` library directories were added to `LD_LIBRARY_PATH` — roughly halved epoch time (~15s → ~10s on the oversampled v3/v4 datasets). Not persisted between shells; re-export before training if picking this back up.
- **Module collision:** this conda env has an unrelated pip package literally named `data`, which was shadowing the local `data/` package under Python's namespace-package resolution rules. Fixed with an empty `data/__init__.py` — harmless, but flagging since it wasn't part of the original plan.
- **coremltools install failed** on this machine due to unrelated, pre-existing corruption in the env (missing `tqdm` dist-info metadata) — `dist/shortcut.json` and `dist/confidence.json` were regenerated directly since they don't depend on the model conversion step; the

TFLite/CoreML model files themselves are untouched from the prior export.

Roman2Khmer experiment report — generated during a Claude Code session

khmerlang/Khmerlang-KeyBoard-IOS · ml/roman2khmer/reports/